Documentação das alterações da GJ00-45: leitura dos dados via API no notebook

Notebook de referência: COPIA_FINAL_API (Google Colab, 00.dev). Alterações de 21/10/2026. Responsável: Luan.

1. Resumo

A task GJ00-45 substituiu a leitura dos dados por arquivos (.zip e .csv do BCB) pela leitura via API (interface de programação de aplicações), com uma chamada por mês. Em paralelo foram corrigidos erros que cortavam os dados do Pix e foram ajustados os cálculos dos Eixos I e II.

Fluxo da execução principal, em cadeia:

`processar()` > `ler_scr` > `ler_pix` > `ler_urls_ibge` > `construir_df_scr_pix` > `validar_scr_contra_origem` > `construir_df_ibge` > `calcular_eixo_i` > `calcular_eixo_ii`

Resultado final conferido (sessão reiniciada e notebook executado do início ao fim):

Conferência	Resultado
SCR (`scr_data.parquet`)	177.653 linhas, 69 meses (202101 a 202609)
Pix (`pix_transacoes.parquet`)	395.447 linhas, 71 meses (202011 a 202609)
Eixo I	1.863 linhas (27 UFs x 69 meses, 202101 a 202609)
Eixo II	1.917 linhas (27 UFs x 71 meses, 202011 a 202609)
Chaves duplicadas e NaN	0 nos dois eixos
Validação de saldos do SCR	soma da origem = soma do `scr_pix` em 69 meses

Convenção de comentários no código. Todo código novo ou alterado leva um destes marcadores: `[NOVO]` (linha criada), `[ALTERADO]` (linha modificada, com o valor antigo quando útil), `[IGUAL]` (linha mantida) e `[REMOVIDO]` (trecho retirado). Por decisão do Luan, os comentários `[REMOVIDO]` foram **mantidos** no código para facilitar a leitura.

2. Fontes de dados

Fonte	Endpoint (ponto de acesso)	Observação
SCR (Banco Central, Olinda)	`https://olinda.bcb.gov.br/olinda/servico/scr_sub_regiao/versao/v1/odata`, recurso `scr_sub_regiao`, parâmetro `DataBase` (AAAAMM)	Um mês por chamada
Pix (Banco Central, Olinda)	`https://olinda.bcb.gov.br/olinda/servico/Pix_DadosAbertos/versao/v1/odata`, recurso `TransacoesPixPorMunicipio`	Um mês por chamada
IBGE (SIDRA)	`https://apisidra.ibge.gov.br`, tabelas 7138 (taxa de escolarização), 4709 (censo demográfico) e 9514 (população por idade)	Linhas com `uf` e `regiao` nulas são o cabeçalho do SIDRA e, no censo, Brasil e regiões (inofensivo)

As fontes acima foram extraídas das URLs configuradas na célula de Configurações do notebook.

3. Mudanças principais

3.1 Pix (prioridade)

Problema. A API do Pix devolve o mês pedido **mais os meses seguintes**, em ordem qualquer. Como a URL limitava o resultado com `$top=35000`, o corte atingia a maior parte do mês pedido. Resultado: apenas 112.139 linhas, com dados cortados.

Agora vêm completos (395 mil). Informar também que o Eixo II ganhou dados completos por consequência.

Correção, em dois pontos:

1. Na URL (célula de Configurações), foi acrescentado `$orderby=AnoMes`, para que a API devolva as linhas em ordem de mês antes do corte:

```
# [ALTERADO] Adicionado "&$orderby=AnoMes"
URL_PIX_API = f"...TransacoesPixPorMunicipio(DataBase=@DataBase)?
@DataBase='<ANO_MES>'&$top=
{PIX_LIMITE_DADOS}&$orderby=AnoMes&$format=json&$select=..."
```

2. Em `processar_url_pix`, o mês pedido é lido da própria URL e o resultado é filtrado para conter só esse mês:

```
ano_mes = url.split("@DataBase='")[1][:6]          # [NOVO] mês que esta
chamada pediu
...
return df[df["AnoMes"].astype(str) == ano_mes]      # [NOVO] descarta os
meses extras
```

Resultado. 395.447 linhas (antes 112.139), 71 meses (202011 a 202609), cerca de 5.570 municípios por mês.

Outras alterações no Pix:

- `tratamento_pix` estava comentada e foi reativada, pois `ler_pix` depende dela (limpeza de nulos e de espaços, conversão numérica, conversão do código IBGE para a sigla da UF e criação da região).
- `ler_pix` passou a usar `ThreadPoolExecutor(max_workers=4)` (conjunto de threads com no máximo 4 chamadas simultâneas), igual ao SCR, para não sobrecarregar a API do BCB.
- `gerar_anos_meses_filtrado` ganhou o parâmetro `inicio` (padrão `2020-11-01`, o que mantém o Pix igual) e passou a ser usada também pelo SCR (início em `2021-01-01`). A função gera a lista de meses AAAAMM e descarta o mês corrente.
- `padronizar_df_pix` (formato longo): transforma as seis colunas do Pix (`VL_PagadorPF`, `QT_PagadorPF`, `VL_PagadorPJ`, `QT_PagadorPJ`, `QT_PES_PagadorPF`, `QT_PES_PagadorPJ`) em linhas, por meio de `melt` (transformação de colunas em linhas). Cada linha passa a ter `tipo` (pf ou pj) e `metrica` (`vl_pagador`, `qt_pagador` ou `qt_pes_pagador`), e os valores são somados por mês, região e UF. Existia uma definição duplicada dessa função (seções Refatoração e Scoring); a cópia da Refatoração foi apagada, pois a da seção Scoring a sobrescrevia.

3.2 Eixo I, Risco de Crédito (prioridade)

O Eixo I gera quatro indicadores por UF e por mês (função `calcular_eixo_i`).

Indicador	Fórmula
`inadimplenciaReal`	`carteira_vencida / carteira_ativa` (0 se a carteira ativa for 0)
`fragilidadeRenda`	carteira ativa da classe de risco `D-H` dividida pela carteira ativa total

Indicador	Fórmula
`agingDivida`	crescimento da carteira vencida em relação ao mesmo mês de 12 meses antes (`PERIODOS_CRESCIMENTO = 12`); 0 quando não há base de comparação
`vulnerabilidadeSocial`	`1 - taxa_escolarizacao`

Normalização (notas de 1 a 5). A função `normalizacao` transforma cada indicador em nota de 1 a 5, **dentro de cada mês**: `nota = 1 + (valor - mínimo) / (máximo - mínimo) x 4`, arredondada em 2 casas. Em cada mês, a UF com o maior valor recebe 5 e a de menor valor recebe 1. Se todas as UFs têm o mesmo valor no mês, a divisão é 0/0 e resulta em NaN, que a limpeza final converte em 0.

Alteração 1: classe `D-H` no lugar de `D` e `E` (decisão de 21/10/2026). A API do SCR traz **risco de crédito** (`AA-C`, `D-H`, `N/D`) e não mais a classe de renda (A a E). Por isso `D-H` passou a valer no lugar de `D` e `E`. O indicador mede agora a fração da carteira ativa em risco alto. Os nomes `carteira_ativa_classes_de` e `fragilidadeRenda` foram mantidos para não quebrar o restante do código.

```
agrupar_metrica(
    df_calculos, "carteira_ativa", df_calculos["classe"].isin(["D-H"])   #
[ALTERADO] antes: .isin(["D", "E"])
).rename("carteira_ativa_classes_de")
```

Alteração 2: marcador 0 para "sem dado" (decisão de 21/10/2026). Entre 202101 e 202412 a API traz `AA-C` e `D-H`. A partir de 202501 ela envia **apenas `N/D`**, portanto não existe `D-H`. Nesses meses todas as UFs ficam com fração 0, a normalização gera NaN (0/0) e a limpeza final converte em 0. Como a escala válida vai de 1 a 5, o **0 significa "sem dado" e não "risco nulo"**. Foi mantido o 0 (sem `ffill` do último valor) porque um NaN seria apagado por dois `fillna(0)` e quebraria o JSON do envio, e carregar o valor de 202412 criaria um dado que a API não publicou. A decisão está registrada em comentário no código. Efeito: 20 meses (202501 a 202608) ficam com `fragilidadeRenda = 0`.

Alteração 3: cobertura completa de anos (decisão de 21/10/2026). Antes, o Eixo I cobria só 2023 a 2025, pois o `merge` (junção) com o IBGE era `inner` (só linhas presentes nos dois lados). Foram feitas três edições em `calcular_eixo_i`:

```
how="left",                                                    # [ALTERADO]
antes: "inner"

df_calculos = df_calculos.sort_values(["uf", "ano_mes"])      # [NOVO] ordena
antes do ffill/bfill

df_calculos = df_calculos.dropna(subset=["carteira_ativa"])   # [NOVO] remove
meses só de Pix
```

- `how="left"` (junção que preserva todas as linhas do SCR) mantém 2021, 2022 e 2026, mesmo sem escolarização do IBGE.
- `sort_values` é necessário porque o `ffill().bfill()` (preenchimento para frente e para trás) depende da ordem das linhas. Sem ordenar, 2021 e 2022 receberiam a escolarização de 2025 em vez da de 2023, e as linhas de Pix e de SCR do mesmo mês ficariam com valores diferentes (o que duplicaria linhas). A diferença foi demonstrada com dados fictícios.
- `dropna` descarta os meses que só existem no Pix (202011, 202012), que apareceriam com zeros falsos.

Aproximações aceitas: 2021 e 2022 usam a escolarização de 2023, e 2026 usa a de 2025 (`vulnerabilidadeSocial` aproximado); o `agingDivida` de 2021 é 0, pois não existe SCR de 2020 para comparar.

Resultado: o Eixo I passou de 972 linhas (202301 a 202512) para 1.863 linhas (202101 a 202609). O valor 1.836 (27 x 68) foi o esperado até a API publicar o mês 202609.

3.3 Eixo II, Inclusão e Expansão Demográfica (prioridade)

O Eixo II gera quatro indicadores por UF e por mês (função `calcular_eixo_ii`).

Indicador	Fórmula
`maturidadePix`	`(uso x 0,6) + (financeira x 0,4)`, onde uso = `qt_pagador / qt_pes_pagador` e financeira = `vl_pagador / qt_pagador` (0 quando o denominador é 0)
`crescimentoPopulacional`	taxa de crescimento geométrico do censo
`totalHabitantes`	população residente do censo
`bonusDemografico`	idade média máxima do ano menos a idade média da UF (calculada em `calculo_idade`, com idade ponderada pela população)

Todos são normalizados em notas de 1 a 5 por mês, como no Eixo I.

O que mudou no Eixo II. O código da função **não foi alterado nesta task**. O Eixo II foi beneficiado pela correção do Pix (dados completos: 395.447 linhas em vez de 112.139) e usa `left` no `merge` com o IBGE, por isso já cobria todo o período (202011 a 202609, 1.917 linhas = 27 x 71). O comentário sobre a fórmula de maturidade (GJ00-33, Miguel) já existia e não faz parte da GJ00-45.

Incidente resolvido. Durante a execução, o `dropna(subset=["carteira_ativa"])` foi colado por engano também em `calcular_eixo_ii`, onde a coluna não existe, gerando `KeyError`. Ele pertence somente à `calcular_eixo_i`. A linha foi removida do Eixo II.

3.4 SCR

Leitura. Antes: download de um .zip por ano e leitura do CSV em pedaços. Agora: uma chamada JSON por mês (`ler_scr` e `processar_url_scr`), desde 202101, com `max_workers=4`. Cada chamada usa `ler_json`, que usa `baixar_com_retry` (leitura com nova tentativa, até `MAX_GET_RETRIES = 5`). Se um mês falha, a mensagem identifica o mês e os demais continuam.

Campos da API e nomes usados no notebook (dicionário `RENAME_SCR`):

Campo da API	Coluna do notebook	Observação
`DATA_BASE`	`data_base`	Vem como número (202401) e vira texto
`CLIENTE`	`cliente`	PF ou PJ
`ESTADO`	`uf`	Sigla da UF
`SUB_REGIAO`	`sub_regiao`	Mantida conforme a task
`MODALIDADE`	`modalidade`	
`RISCO`	`classe`	Substitui o antigo `porte`; traz `AA-C`, `D-H`, `N/D`
`CARTEIRA`	`carteira_ativa`	
`VENCIDO_ACIMA_DE_15_DIAS`	`carteira_vencida`	Significado ainda a confirmar (pergunta 1)

A API não possui colunas de vencido por faixa (15 a 60, 61 a 90 e acima de 90 dias).

Tratamento (`tratamento_scr_api`), em cadeia: renomear campos > limpar espaços > `classe` nula vira `N/D` > `data_base` vira texto > remover `uf = NI` (estado não identificado, que não existe no mapa de regiões) > criar `regiao` a partir da `uf` > garantir tipo numérico > valores ausentes viram 0. Os dados são salvos em `scr_data.parquet`.

Padronização (`padronizar_df_scr`). Transforma `carteira_vencida` e `carteira_ativa` em linhas (`metrica`), cria `tipo` (pf ou pj), mantém `modalidade` e `classe` e agrupa os valores. A coluna `sub_regiao` passou a ser **mantida** no resultado (item 4 do checklist): entrou no `melt`, na conversão para categoria, no `groupby` e no `return`, e também no `groupby` de `construir_df_scr_pix`. As linhas do Pix recebem `sub_regiao` vazia. Os Eixos I e II somam por mês, região e UF, portanto não foram afetados.

Validação de saldos. A função antiga `validar_consistencia_saldos` comparava a soma do dado com a soma do mesmo dado (sempre "100% consistente") e rodava por pedaço de 100 mil linhas, repetindo a mensagem. A chamada foi substituída por comentário `[REMOVIDO]` e foi criada `validar_scr_contra_origem()`, que compara, por mês, a soma de `carteira_ativa` e `carteira_vencida` do parquet bruto da API com a do `scr_pix.parquet` final (tolerância de R$ 1). Roda uma vez, depois de `construir_df_scr_pix()`.

3.5 IBGE

- Censo demográfico e idade só existem em anos de censo (2010 e 2022). Por isso, `construir_df_ibge` fica com o dado mais recente de cada UF (`.last()`), retira a coluna `ano` e cruza por `regiao` e `uf` (antes cruzava por `ano`, `regiao` e `uf`).
- A escolarização é a média por ano, região e UF (anos disponíveis: 2023, 2024 e 2025).
- `.reset_index()` virou `.reset_index(drop=True)`. Sem o `drop=True`, a numeração antiga das linhas virava a coluna `index` no `ibge.parquet`, que nenhum código usa. Conferido: a coluna deixou de existir.

4. Como os dados foram transformados para serem exibidos

Cadeia geral, da fonte ao resultado:

API (JSON) > DataFrame (tabela de dados) > tratamento > arquivo parquet (formato colunar de arquivo) > formato longo unificado (`scr_pix`) > cálculo por eixo > normalização de 1 a 5 > limpeza final

Arquivos gerados em `/content/dados`:

Arquivo	Conteúdo
`scr_data.parquet`	SCR tratado (uma linha por mês, UF, sub-região, cliente, modalidade e classe)
`pix_transacoes.parquet`	Pix por município, já filtrado para o mês pedido
`taxa_escolarizacao.parquet`, `censo_demografico.parquet`, `populacao_por_idade.parquet`	Tabelas do IBGE tratadas
`scr_pix.parquet`	SCR e Pix no **formato longo**: `ano_mes`, `regiao`, `uf`, `sub_regiao`, `tipo`, `classe`, `metrica`, `origem`, `valor`
`ibge.parquet`	Escolarização, população, taxa de crescimento, variação da população e bônus demográfico por UF

Formato longo. Pix e SCR deixam de ter uma coluna por indicador e passam a ter uma linha por indicador. A coluna `metrica` identifica o que cada valor representa (`carteira_ativa`, `carteira_vencida`, `vl_pagador`, `qt_pagador`, `qt_pes_pagador`) e `origem` indica `scr` ou `pix`. As linhas do Pix não têm `classe` nem `sub_regiao`.

Do `scr_pix` ao resultado exibido. Cada eixo soma as métricas por `ano_mes`, `regiao` e `uf` (`agrupar_metrica`), junta o IBGE, calcula os indicadores, normaliza em notas de 1 a 5 e devolve as colunas finais:

Eixo	Colunas finais
Eixo I	`ano_mes`, `regiao`, `uf`, `inadimplenciaReal`, `fragilidadeRenda`, `agingDivida`, `vulnerabilidadeSocial`
Eixo II	`ano_mes`, `regiao`, `uf`, `maturidadePix`, `crescimentoPopulacional`, `totalHabitantes`, `bonusDemografico`

Limpeza antes do envio (execução principal): infinitos e NaN viram 0 nos dois eixos; as colunas categóricas do `scr_pix` viram texto (`object`) para que o `fillna(0)` funcione (o 0 não pode entrar em uma categoria que não existe), e o campo `bonusDemografico` é retirado do `df_ibge`.

Como ler as notas. As notas de 1 a 5 são **relativas ao mês**: comparam as UFs entre si naquele mês e não são um valor absoluto. O **0 está fora da escala** e indica ausência de dado (por exemplo, `fragilidadeRenda` de 202501 em diante).

5. Alterações secundárias

Alteração	Estado
Constantes e URLs da API na célula de Configurações (`URL_BASE_SCR` , `ENDPOINT_SCR` , `RENAME_SCR` , `CAMPOS_API_SCR` , `URL_SCR_MES` , `PERIODOS_CRESCIMENTO`)	Feito
Remoção da leitura por .zip e .csv (restam só comentários `[REMOVIDO]`)	Feito
Salvamento dos parquets com `salvar_parquet`	Feito
Diagnóstico temporário da execução principal apagado	Feito
`ThreadPoolExecutor(max_workers=4)` no Pix	Feito
`reset_index(drop=True)` no IBGE	Feito
Substituição da validação de saldos por `validar_scr_contra_origem`	Feito
`NIVEIS_SALARIO` e `converter_nivel_salario` (código morto, nunca chamado): substituir por comentário `[REMOVIDO]`	Feito

6. Decisões registradas

Data	Decisão
21/10/2026	`D-H` substitui `D` e `E` no `fragilidadeRenda`
21/10/2026	Manter 0 como marcador de "sem dado" no `fragilidadeRenda` (sem `ffill`)
21/10/2026	Aplicar a cobertura completa de anos no Eixo I (escolarização emprestada em 2021, 2022 e 2026)
21/10/2026	Manter os comentários `[REMOVIDO]` no código

Data	Decisão
21/10/2026	Alterações de código no chat: só o código e o local; explicações na documentação

7. Limitações e pontos de atenção

- A validação de saldos compara o parquet baixado da API com o `scr_pix`. Ela detecta perdas e erros do próprio notebook, mas **não** erros na origem (a API).
- A execução principal **não chama `enviar`** e a `URL_API_LOCAL` (túnel) está fixa no código.
- O `pd.concat` de `construir_df_scr_pix` emite um `FutureWarning` por causa das colunas vazias `classe` e `sub_regiao` do Pix. É inofensivo e não altera o resultado.
- A API do SCR passou a publicar o mês 202609 durante a sessão (o SCR foi de 176.035 para 177.653 linhas). O valor de referência do Eixo I mudou de 1.836 para 1.863 por esse motivo, e não por edição de código.
- Valores de referência dependem do que a API publica no dia da execução.